Assignment No: 3

Title: Apply CSS (Inline, Internal, External) to style a webpage.

Aim: To learn and demonstrate the three fundamental methods of applying Cascading Style Sheets (CSS) - Inline, Internal and External - to style a webpage, and to understand how styling rules interact with HTML elements.

Objectives:

1. To learn and implement all three methods of applying CSS.
2. To understand the difference between inline, internal and external CSS.
3. To learn how to separate webpage content from visual design.
4. To understand the use of colors, fonts, spacing and contrast in webpage design.
5. To develop organized and scalable CSS code for webpages.
6. To understand how CSS rules are applied and prioritized by the browser.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS

Theory:

1. Introduction to CSS

Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation, layout and visual formatting of a document written in HTML. HTML provides the structure of a webpage; CSS styles those elements by defining colors, fonts, spacing, alignment, borders and positioning.

2. Inline CSS

Inline CSS is written directly inside an individual HTML element using the style attribute. It is used when a specific style must be applied to a single element, and it is the only method that can take a value computed at runtime.

Syntax:

```
<p style="color: blue; font-size: 16px;">This is a paragraph.</p>
```

Advantages: easy to apply to a single element; useful for quick styling and for dynamic values. Disadvantages: difficult to maintain, repeats styling rules and mixes content with presentation.

3. Internal CSS

Internal CSS, also called embedded CSS, is written inside a <style> tag. It is useful when styling a single webpage with multiple elements, and it is the only way to define an @keyframes animation whose values depend on the current page.

Syntax:

```
<head>
  <style>
    p { color: green; font-size: 18px; }
  </style>
</head>
```

Advantages: styles multiple elements on one webpage and keeps the rules inside the document. Disadvantages: the styles are not shared with other pages and the HTML file becomes longer.

4. External CSS

External CSS is written in a separate file with a .css extension and linked to the HTML document using the <link> tag inside the <head> section.

HTML file:

```
<link rel="stylesheet" href="style.css">
```

CSS file (style.css):

```
p {
  color: blue;
  font-size: 18px;
}
```

Advantages: separates HTML content from CSS design, allows one file to style many pages, is easy to maintain and reduces repeated code.

5. Difference Between Inline, Internal and External CSS

Feature	Inline CSS	Internal CSS	External CSS
Location	Inside HTML element	Inside <style> tag	Separate .css file
Scope	Single element	One HTML document	Multiple webpages
Reusability	Low	Limited	High
Maintenance	Difficult	Moderate	Easy
Best use	Quick / dynamic styling	Single webpage	Large websites

6. CSS Syntax, Cascade and Priority

CSS consists of a selector, a property and a value. When multiple CSS rules apply to the same element, the browser decides which rule wins using the cascade: the importance of the rule, the specificity of the selector, and the source order when the priority is equal. Inline styles generally have higher priority than ordinary internal and external styles.

Example:

```
h1 {
  color: blue;    /* property : value */
  font-size: 24px;
}
```

Course Project Implementation:

Course-Project Mapping: Fully available in the course project. LipiSutra uses all three methods at the same time, which makes it an unusually complete example of this assignment. Submit src/index.css as the external stylesheet, src/App.jsx for inline styling, and src/components/UploadSection.jsx for internal styling.

CSS method	File to submit	Evidence
External	src/index.css	78-line stylesheet imported by src/main.jsx
Inline	src/App.jsx	style={{ backgroundColor, color, transition }} on the root div
Internal	src/components/UploadSection.jsx	a literal <style> element holding the @keyframes pulse-spin rule

The internal <style> block is used in UploadSection.jsx for a genuine engineering reason: the keyframes interpolate the current theme accent colour, which is only known at runtime, so the rule cannot live in the static external stylesheet.

Program:

1. EXTERNAL CSS - src/index.css (excerpt)

```
@import url('https://fonts.googleapis.com/css2?
family=Playfair+Display:wght@400..900&family=Inter:wght@300;400;600;800&display=swap')
;
@import "tailwindcss";

@theme {
  --font-heading: 'Playfair Display', serif;
  --font-sans:    'Inter', sans-serif;

  --color-museum-900: #140c05;
  --color-museum-800: #1a120b;
  --color-gold-400:   #E5C366;
  --color-gold-600:   #8B6914;
  --color-slate-200:  #f5e6c8;

  @keyframes fadeIn {
    from { opacity: 0; transform: translateY(10px); }
    to   { opacity: 1; transform: translateY(0); }
  }
}

@layer base {
  body {
    background-color: var(--color-museum-900);
    background-size: cover;
    background-position: center;
    background-attachment: fixed;
    color: var(--color-slate-200);
    -webkit-font-smoothing: antialiased;
    min-height: 100vh;
  }
  h1, h2, h3, h4, .font-heading { color: #C9A84C; }
}
```

```
.glass-card {
  background: rgba(26, 18, 11, 0.6);
  backdrop-filter: blur(8px);
  border: 1px solid rgba(201, 168, 76, 0.15);
}
.glass-card:hover {
  border-color: rgba(201, 168, 76, 0.3);
  background: rgba(26, 18, 11, 0.7);
}

::-webkit-scrollbar       { width: 8px; }
::-webkit-scrollbar-track { background: var(--color-museum-900); }
::-webkit-scrollbar-thumb { background: var(--color-gold-900); border-radius: 4px; }
```

Linked from src/main.jsx (the equivalent of <link rel="stylesheet">)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'            // <-- external stylesheet linked here
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

2. INLINE CSS - src/App.jsx (excerpt)

```
<div
  className="min-h-screen relative flex flex-col font-sans"
  style={{
    backgroundColor: theme.bg,
    backgroundSize: 'cover',
    backgroundPosition: 'center',
    backgroundAttachment: 'fixed',
    color: theme.text,
    transition: "background 0.3s, color 0.3s"
  }}
>
  <header className="absolute top-4 right-4 z-[100]">
    <button
      onClick={toggleTheme}
      style={{
        backgroundColor: themeMode === 'LIGHT' ? theme.accent : theme.surface,
        color:           themeMode === 'LIGHT' ? '#fff8f0'    : theme.text,
        border: `1.5px solid ${theme.border}`,
        transition: "all 0.3s"
      }}
    >
      {themeMode === "DARK" ? "Dark" : "Light"}
    </button>
  </header>
```

3. INTERNAL CSS - src/components/UploadSection.jsx (excerpt)

```
<style>
  {`
    @keyframes pulse-spin {
      0%   { transform: rotate(0deg)   scale(0.9);
             box-shadow: 0 0 0  0px ${theme.accent}66; }
      50%  { transform: rotate(180deg) scale(1.1);
             box-shadow: 0 0 0 20px ${theme.accent}00; }
      100% { transform: rotate(360deg) scale(0.9);
             box-shadow: 0 0 0  0px ${theme.accent}00; }
    }
  `}
</style>
```

```
<div style={{
  width: '64px', height: '64px',
  border: `4px solid ${theme.accent}33`,
  borderTopColor: theme.accent,
  borderRadius: '50%',
  animation: 'pulse-spin 1.5s cubic-bezier(0.4, 0, 0.2, 1) infinite'
}}></div>
```

Output:

- The external stylesheet paints the parchment background, the cream body text and the gold headings across every page of the application.
- The inline style on the root swaps the whole palette the instant the Dark/Light button is clicked, because the values are read from the active theme object.
- The internal <style> block drives the gold loading spinner that pulses while a manuscript is being decoded.
- In DevTools > Elements > Computed, the inline declaration is shown overriding the stylesheet rule for background-color, confirming the cascade order taught in the theory.

Conclusion:

In this assignment, we learned and implemented the three methods of applying CSS: inline, internal and external. We understood how CSS is used to style HTML elements and how styling rules are prioritized by the browser cascade. In the LipiSutra project all three methods coexist for well-defined reasons - the external file for the shared design system, inline styles for values computed from the runtime theme, and an internal <style> block for keyframes that depend on that same runtime theme.

References:

7. Mozilla Developer Network (MDN) Web Docs - CSS Basics, https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
8. W3Schools - CSS Tutorial, https://www.w3schools.com/css/
9. CSS Specifications - W3C, https://www.w3.org/Style/CSS/